

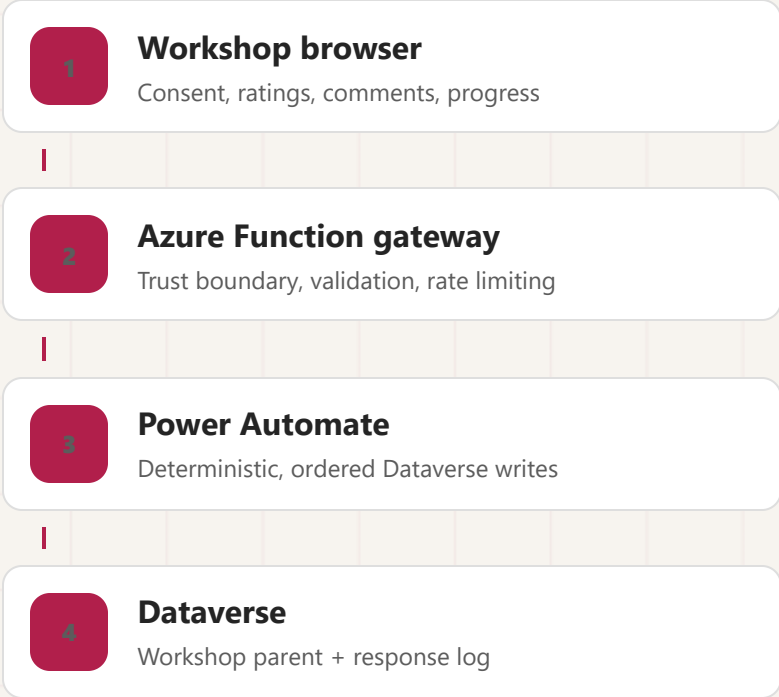
AGENT JUMPSTART SURVEY ARCHITECTURE

From participant feedback to a governed Dataverse record

A step-by-step technical walkthrough of the Azure Function gateway, Power Automate capture flow, Dataverse data model, privacy boundary, failure handling, and operational trace.



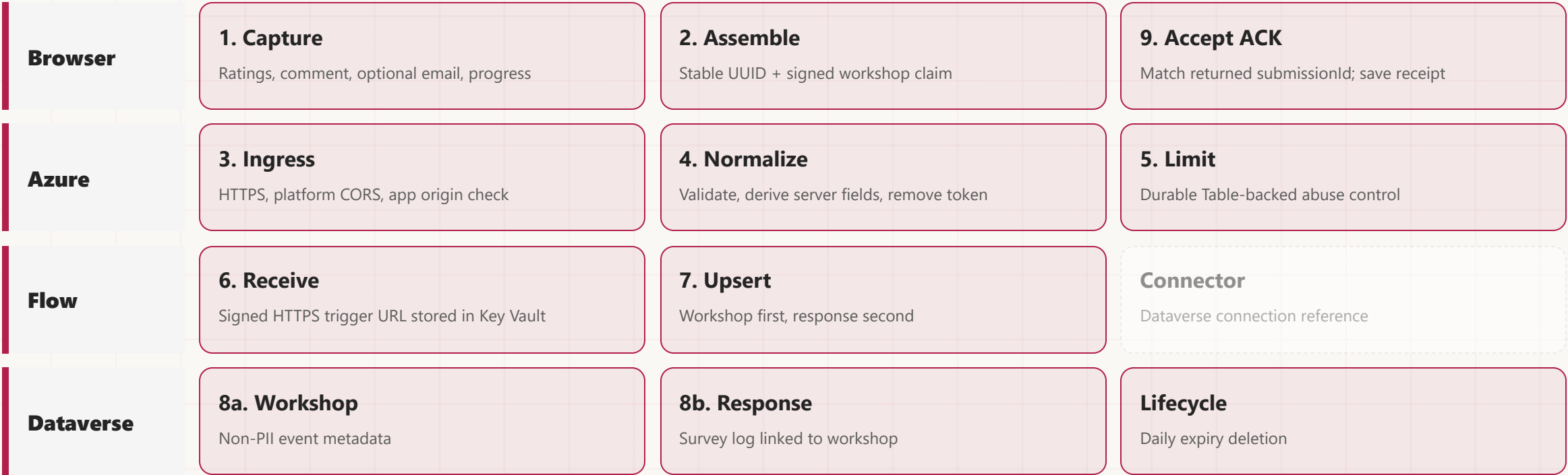
Architecture reference · September 18, 2026



— ARCHITECTURE AT A GLANCE

Four execution zones, **nine ordered stages**

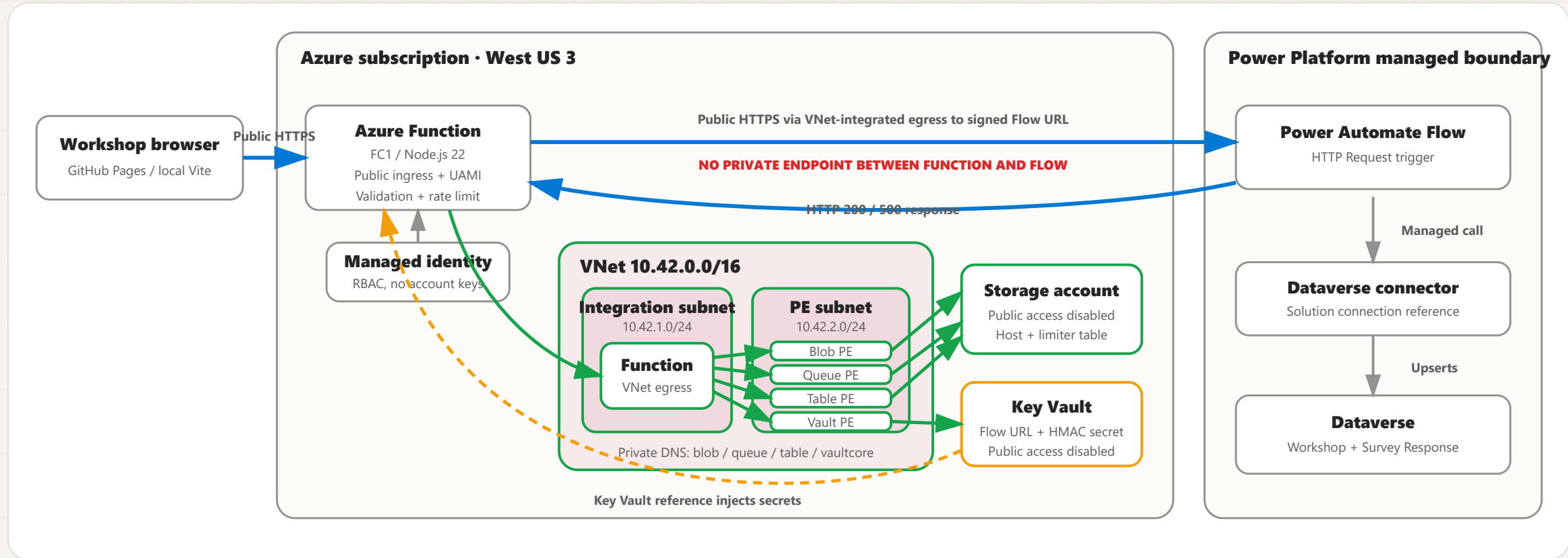
The browser knows only the public gateway. The callback URL, signing secret, connector identity, and persistence operations remain behind managed service boundaries.



Sidecars: Key Vault protects the Flow URL and HMAC secret; managed identity reaches private Storage and Key Vault; Application Insights and Flow run history provide operational evidence.

NETWORK TOPOLOGY • PRIVATE VERSUS PUBLIC PATHS

Two private Azure data paths; one public Flow handoff



Key distinction: the Flow URL is secret, but not a private endpoint. Flex routes outbound through the integration subnet, then reaches the Flow's public HTTPS trigger. Private Link here covers Storage and Key Vault only.

STEP 1 • WORKSHOP BROWSER

Capture useful feedback without **silently collecting identity**

A All labs complete modal

- + Three independent 0-5 ratings: overall, effort, recommend
- + Free-text comment, capped later at 10,000 characters
- + At least one rating or a non-empty comment enables submit
- + Answers remain in React state when a submission fails

B Explicit email consent

- + Email starts excluded, even after workshop access verification
- + Participant must select "Include my email"
- + Without consent the outgoing email is an empty string
- + Privacy copy states the 24-month retention period

Progress snapshot

Each public lab contributes its visible step IDs, completed count, and total count. Private labs and hidden steps are excluded.

`completedSteps``totalSteps``labStatus[]`

Deliberately not submitted

Lab usernames, access codes, attendee allow-list, facilitator contacts, and maker credentials never enter the survey payload.

Boundary: survey collection is separate from workshop access validation.

STEP 2 • BROWSER REQUEST

Resolve public configuration, then build a **retry-safe contract**

Endpoint discovery

```
GET /content/feedback.json
{
  "enabled": true,
  "endpoint": "https://...azurewebsites.net/api/feedback"
}
```

The client rejects disabled, empty, or non-HTTPS remote endpoints. The Power Automate callback is never published.

Stable idempotency key

`crypto.randomUUID()` runs once when the modal mounts. Every click of Retry reuses that same `submissionId`.

POST /api/feedback
Content-Type: application/json

schemaVersion: 1
submissionId: UUID
workshopId: UUID
workshopKey: "workshop:<UUID>"
customerName / hostName
workshopStart / workshopEnd
surveyToken: signed workshop claim
attendeeEmail / emailConsent
locale: one of 9 values
overall / effort / recommend: 0..5
comments
completedSteps / totalSteps
labStatus[]
sourcePageUrl

Client timeout: the browser aborts after 30 seconds. It accepts success only when HTTP is OK and the ACK contains the exact same `submissionId`.

STEP 3 • AZURE INGRESS BOUNDARY

Public HTTPS ingress with **two explicit Origin checks**

1 Azure platform CORS

App Service answers browser preflight only for the configured origins.

- + GitHub Pages origin
- + localhost:5173
- + 127.0.0.1:5173

2 Application Origin check

The Function reads `Origin` and rejects any non-allowlisted value with 403 `ORIGIN_NOT_ALLOWED`.

3 Method boundary

`OPTIONS` returns 204, `POST` enters processing, all other methods return 405.

Why the HTTP trigger is anonymous

GitHub Pages has no server identity. The endpoint therefore uses anonymous Function auth and compensating controls: exact CORS allowlist, signed workshop claim, strict schema validation, request size cap, durable rate limiting, and no direct database credential.

Security nuance

CORS is a browser control, not authentication; callers without an `Origin` header are not blocked by it. The survey token proves workshop-claim integrity, not participant identity. Abuse resistance comes from validation, managed-identity service boundaries, and rate limiting.

STEP 4 • GATEWAY VALIDATION

Reject early; forward only a **normalized survey**

A

Envelope

Maximum 128 KiB, valid JSON object, no malformed transport.

>

B

Identity

UUIDs, workshop key, customer/host text, YYYY-MM-DD dates.

>

C

HMAC claim

Timing-safe comparison over workshop ID, key, names, and dates.

>

D

Survey

Consent/email, locale, ratings, comments, counts, lab status consistency.

9

supported locales

100

maximum lab-status items

100 KB

serialized lab-status cap

Cross-field invariant: the sum of per-lab completed and total steps must exactly match the top-level counts. This prevents a client from sending contradictory progress.

STEP 4B • SERVER-OWNED DATA

The gateway removes trust-sensitive input and **adds authoritative fields**

Browser input

surveyToken
emailConsent + candidate email
labStatus[]
sourcePageUrl
progress counts



Normalized Flow payload

token removed after verification
email blank unless consent = true
labStatusJson serialized
URL constrained to allowlisted origin
completionPercent derived

submittedAt

Generated from server UTC time,
never trusted from the browser.

retentionExpiresAt

Server time plus 24 months.

userAgent

Captured from the HTTP request,
maximum 500 characters.

processingVersion

Stamped as gateway-v1.

Canonical source: sourceSiteUrl comes from Function configuration. A submitted page URL is retained only when its Origin is allowlisted.

STEP 5 • AZURE TABLE RATE LIMITER

Count distinct submissions without storing **raw client IP**

Entity key design

```
PartitionKey = YYYYMMDDHH + "-" +  
base64url(HMAC-SHA256(clientAddress, signingSecret))[0:32]
```

```
RowKey = submissionId
```

A user-assigned managed identity authenticates to the private Table endpoint; no storage key is used.

Decision algorithm

- + Create an entity for this client-hour + submission ID
- + A 409 conflict means this is the same retry; continue
- + Count rows in the partition
- + Allow up to 120 distinct submissions per hour
- + Return 429 + Retry-After: 3600 above the limit

120

distinct IDs / client / hour

48 h

limiter row retention

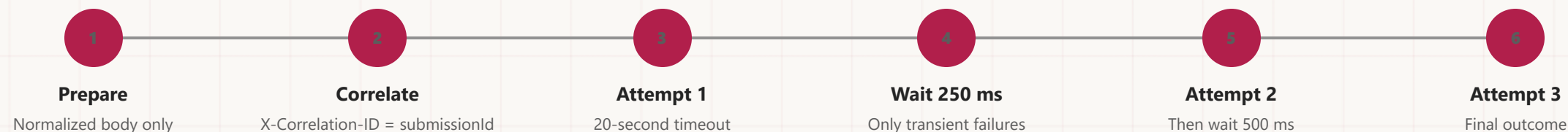
04:15

UTC cleanup timer, daily

Fail closed: if the limiter cannot reach Table Storage, the gateway returns 503 and does not call Power Automate.

STEP 6 • SERVICE-TO-SERVICE HANDOFF

Forward normalized JSON to a **protected HTTPS** callback

**Retry**

Network/timeout exceptions, HTTP 429, and HTTP 5xx responses. Every attempt carries the same submission ID.

Do not retry

Other HTTP 4xx responses indicate a contract or authorization problem. The loop stops immediately.

Secret, not Private Link: SURVEY_FLOW_URL is a Key Vault reference and never reaches the browser, source, or logs. Flex routes the runtime call through VNet-integrated egress to the Power Automate trigger's public HTTPS endpoint; no Function-to-Flow Private Endpoint exists.

STEP 7 • POWER AUTOMATE CAPTURE FLOW

One trigger, two ordered upserts, **one deterministic response**

1

Receive Survey Response

HTTP Request trigger validates the normalized schema; additional properties are disallowed.

2

Upsert Workshop

Dataverse UpdateRecord using workshopId as the deterministic row ID.

3

Upsert Survey Response

Uses submissionId; binds mjsrc_WorkshopId to the parent.

4

Return

200 with matching ID after both writes; 500 when the response write fails, times out, or is skipped.

Connection

shared_commondataserviceforapps through a solution connection reference.

Order

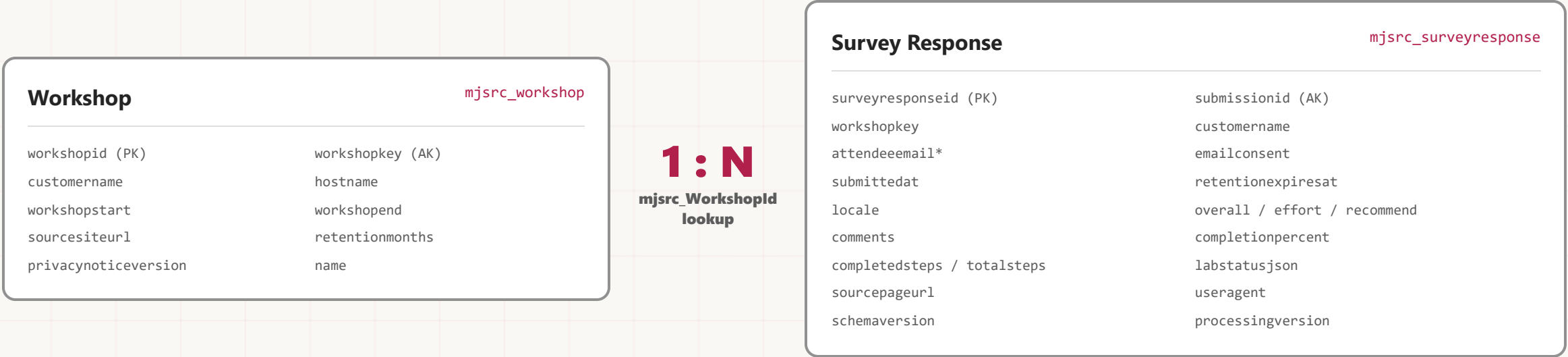
Response write runs only after Workshop succeeds, ensuring the lookup target exists.

Replay

Deterministic row IDs make Function retries update the same records instead of appending duplicates.

STEP 8 • DATAVERSE RECORD MODEL

Workshop metadata once; responses as the durable survey log



* PII rule: attendeeemail is populated only when emailConsent = true; otherwise Flow writes an empty string. Workshop rows contain no participant PII.

STEP 9 • RETURN PATH

Success means **Dataverse finished**, not merely "request accepted"

1 Flow ACK

After both upserts, Flow returns:

```
HTTP 200
{
  "ok": true,
  "submissionId": "..."
}
```

2 Gateway ACK

The Function treats any HTTP 2xx Flow response as stored, logs the submission ID, and returns its own matching 200 ACK.

3 Browser commit

The UI shows Thanks and saves a local feedback receipt only after `ok = true` and exact ID equality.

Browser layer

One UUID per modal session; retry reuses it.

Limiter layer

Duplicate RowKey conflict is accepted and not counted twice.

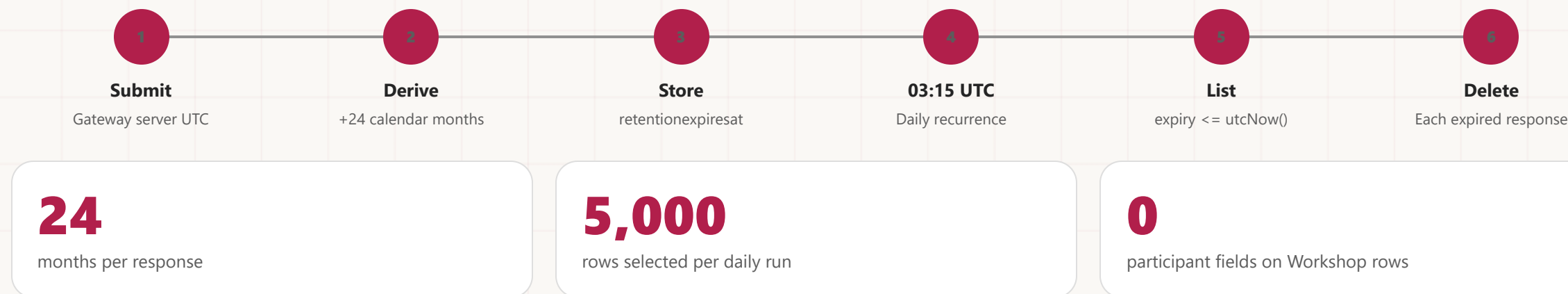
Persistence layer

GUID row IDs update the same Workshop and Survey Response.

Net effect: retries can safely cross browser, Function, and Flow boundaries without creating duplicate survey rows.

LIFECYCLE · RETENTION FLOW

Expiry is assigned at ingestion and **enforced every day**



Operational consideration: the current Flow selects at most 5,000 expired rows per run. If expiry volume can exceed that daily capacity, add pagination or repeated batches.

FAILURE SEMANTICS

Every rejection has a layer, code, and **write guarantee**

HTTP	Code	Where	Meaning / persistence outcome
400	INVALID_JSON	Function	Body cannot be parsed; Flow is not called.
400	INVALID_SURVEY	Function	Field, consent, rating, date, locale, or progress invariant failed; no write.
403	ORIGIN_NOT_ALLOWED	Function	Application Origin allowlist rejected the caller; no write.
403	INVALID_WORKSHOP_TOKEN	Function	Signed workshop claim mismatch; no write.
413	SURVEY_TOO_LARGE	Function	Request exceeds 128 KiB; no write.
429	SURVEY_RATE_LIMITED	Function	More than 120 distinct IDs in the client-hour partition; no Flow call.
503	SURVEY_UNAVAILABLE	Function	Required configuration or rate limiter unavailable; fail closed.
502	SURVEY_FORWARD_FAILED	Function	Flow still failed after retry policy; browser keeps answers for retry.
500	SURVEY_STORAGE_FAILED	Flow	Dataverse response write failed/timed out/skipped; surfaced to Function and retried when transient.

OPERATIONS · CORRELATION

Trace one response end to end with **submissionId**

1. Browser

submissionId generated once local receipt only after ACK

2. Function

X-Correlation-ID
"Survey response stored" trace

3. Flow

Run history trigger body
action outputs / failure branch

4. Dataverse

mjsrc_submissionid
response PK = submissionId

5. Parent

mjsrc_WorkshopId
workshop PK = workshopId

Function health

GET /api/health reports presence of Flow, signing, and rate-limit configuration.

Azure telemetry

Application Insights receives traces through public Azure Monitor ingestion via VNet-integrated egress; no AMPLS/private endpoint is configured.

Data verification

Query Survey Response by mjsrc_submissionid, then follow the Workshop lookup.

Do not search by email first: email may be intentionally absent. The submission ID is the stable correlation key across every layer.

REFERENCE INVENTORY

Deployed services, ownership boundaries, and **repeatable checks**

Azure / Power Platform inventory

- + Resource group azrge6s5zfqaajgpoo, West US 3
- + Function azfnuhqsqqzypov3s, FC1, Node.js 22
- + User-assigned identity; private Storage + Key Vault endpoints
- + Application Insights + Log Analytics diagnostics
- + Solution AgentJumpStartSurvey
- + Capture Flow + daily retention Flow
- + Tables mjsrc_workshop and mjsrc_surveyresponse

Validation sequence

```
npm run build
npm test --prefix feedback/azure-function
az deployment sub validate ...
node scripts/test_survey_gateway.mjs
node scripts/test_survey_gateway.mjs --invalid-token
```

Positive smoke records use deterministic synthetic IDs and must be removed after validation. Negative-HMAC tests must leave Dataverse unchanged.

Secrets

Flow URL + HMAC secret remain in Key Vault references.

Credentials

Managed identity accesses Azure data services; Flow owns the Dataverse connector.

Source of truth

Bicep, Flow definitions, solution XML, and tests live in this repository.

Design principle: the public client can submit a narrowly defined survey, but it cannot discover the Flow callback, authenticate to Storage, or write Dataverse directly.